A Comprehensive Study on Neural Networks and Deep Learning Architectures”

Index

1. Introduction

What is a Neural Network?

A neural network is a type of machine learning algorithm inspired by the human brain. It's powerful tool that excels at solving complex problems more difficult for traditional computer algorithms to handle, such as image recognition and natural language processing.

Composed of interconnected nodes called neurons, neural networks arrange these units in layers. Each neuron receives input from others, processes it, and transmits an output to other neurons. Connections between neurons have associated weights, signifying the connection strength. During training, the network adjusts these weights to refine its performance on a given task. This learning process allows them to make predictions and recognize patterns, driving their wide adoption in diverse applications like image recognition, natural language processing, and machine translation.

Understanding Neural Networks in Deep Learning

Neural networks are capable of learning and identifying patterns directly from data without predefined rules. These networks are built from several key components:

- Neurons: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- Connections: Links between neurons that carry information, regulated by weights and biases.
- Weights and Biases: These parameters determine the strength and influence of connections.
- Propagation Functions: Mechanisms that help process and transfer data across layers of neurons.
- Learning Rule: The method that adjusts weights and biases over time to improve accuracy.

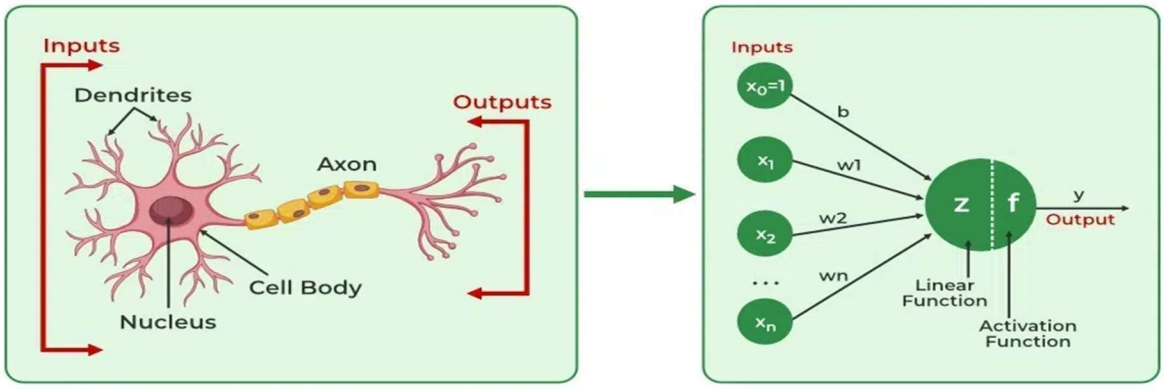
Learning in neural networks follows a structured, three-stage process:

- Input Computation: Data is fed into the network.
- Output Generation: Based on the current parameters, the network generates an output.
- Iterative Refinement: The network refines its output by adjusting weights and biases, gradually improving its performance on diverse tasks.

In an adaptive learning environment:

- The neural network is exposed to a simulated scenario or dataset.
- Parameters such as weights and biases are updated in response to new data or conditions.

With each adjustment, the network's response evolves, allowing it to adapt effectively to different tasks or environments.



Importance of Neural Networks

Neural networks are pivotal in identifying complex patterns, solving intricate challenges, and adapting to dynamic environments. Their ability to learn from vast amounts of data is transformative, impacting technologies like natural language processing, self-driving vehicles, and automated decision-making. Neural networks streamline processes, increase efficiency, and support decision-making across various industries. As a backbone of artificial intelligence, they continue to drive innovation, shaping the future of technology.

2. Evolution of Neural Networks

Neural networks have undergone significant evolution since their inception in the mid-20th century. Here's a concise timeline of the major developments in the field:

1940s-1950s: The concept of neural networks began with McCulloch and Pitts' introduction of the first mathematical model for artificial neurons. However, the lack of computational power posed significant challenges to further advancements.

1960s-1970s: Frank Rosenblatt's worked on perceptions. Perceptions are simple single layer networks that can solve linearly separable problems but can not perform complex tasks.

1980s: The development of backpropagation by Rumelhart, Hinton, and Williams revolutionized neural networks by enabling the training of multi-layer networks. This period also saw the rise of connectionism, emphasizing learning through interconnected nodes.

1990s: Neural networks experienced a surge in popularity with applications across image recognition, finance, and more. However, this growth was tempered by a period known as the "AI winter," during which high computational costs and unrealistic expectations dampened progress.

2000s: A resurgence was triggered by the availability of larger datasets, advances in computational power, and innovative network architectures. Deep learning, utilizing multiple layers, proved highly active across various domains.

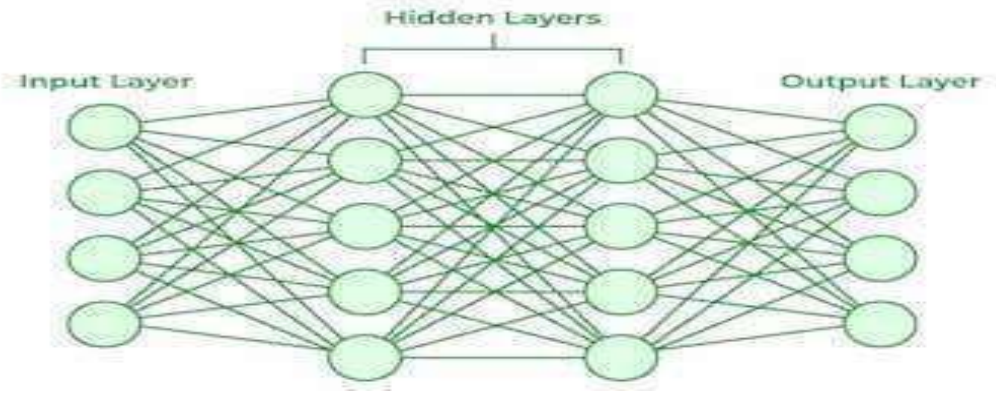
2010s: The landscape of machine learning has been dominated by deep learning with CNNs (Convolutional Neural Networks) excelling in image classification and RNNs (Recurrent Neural

Networks), LSTMs, and GRUs gaining traction in sequence-based tasks like language modeling and speech recognition.

2017: Transformer models, introduced by Vaswani et al. in "Attention is All You Need," revolutionized NLP by using a self-attention mechanism for parallel processing, improving efficiency. Models like BERT, GPT, and T5 set new benchmarks in machine translation and text generation.

3. Neural Network Architecture

1. Input Layer: This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
2. Hidden Layers: These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.
3. Output Layer: The final layer produces the output of the model. The format of these outputs varies depending on the specific task (e.g., classification, regression).



Working of Neural Networks Forward Propagation

When data is input into the network, it passes through the network in the forward direction, from the input layer through the hidden layers to the output layer. This process is known as forward propagation. Here's what happens during this phase:

1. Linear Transformation: Each neuron in a layer receives inputs, which are multiplied by the weights associated with the connections. These products are summed together, and a bias is added to the sum. This can be represented mathematically as:
   $Z = w1x1+w2x2+\ldots+wnxn+b$ where w represents the weights, x represents the inputs, and b is the bias.
2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. The activation function is crucial because it introduces non-linearity into the

system, enabling the network to learn more complex patterns. Popular activation functions include ReLU, sigmoid, and tanh.

Working of Neural Networks Backpropagation Propagation

After forward propagation, the network evaluates its performance using a loss function, which measures the difference between the actual output and the predicted output. The goal of training is to minimize this loss. This is where backpropagation comes into play:

1. Loss Calculation: The network calculates the loss, which provides a measure of error in the predictions. The loss function could vary; common choices are mean squared error for regression tasks or cross entropy loss for classification.
2. Gradient Calculation: The network computes the gradients of the loss function with respect to each weight and bias in the network. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.
3. Weight Update: Once the gradients are calculated, the weights and biases are updated using an optimization algorithm like stochastic gradient descent (SGD). The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.

Iteration Learning

1. This process of forward propagation, loss calculation, backpropagation, and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss, and the network's predictions become more accurate.

2. Through these steps, neural networks can adapt their parameters to better approximate the relationships in the data, thereby improving their performance on tasks such as classification, regression, or any other predictive modeling.

Example of Email Classification

Let's consider a record of an email dataset:

Email ID	Email Content	Sender	Subject Line	Label

1	"Get free gift cards now!"	spam@example.com	"Exclusive offer"	1

To classify this email, we will create a feature vector based on the analysis of keywords such as "free," "win," and "o er."

The feature vector of the record can be presented as:

- "free": Present (1)
- "win": Absent (0)
- "offer": Present (1)

How Neurons Process Data in a Neural Network

In a neural network, input data is passed through multiple layers, including one or more hidden layers. Each neuron in these hidden layers performs several operations, transforming the input into a usable output.

1. Input Layer: The input layer contains 3 nodes that indicates the presence of each keyword.
2. Hidden Layer: The input data is passed through one or more hidden layers.

Each neuron in the hidden layer performs the following operations

Weighted Sum: Each input is multiplied by a corresponding weight assigned to the connection.

For example, if the weights from the input layer to the hidden layer neurons are as follows: Weights for Neuron H1: [0.5, -0.2, 0.3]

Calculate Weighted Input: o For Neuron H1:

Calculation = $(1\times0.5) + (0\times-0.2) + (1\times0.3) = 0.5 + 0 + 0.3 = 0.8$

Neuron H2:

Calculation = $(1\times0.4) + (0\times0.1) + (1\times-0.5) = 0.4 + 0 - 0.5 = -0.1$

Activation Function: The result is passed through an activation function (e.g., ReLU or sigmoid) to introduce non-linearity.

For H1, applying ReLU: ReLU(0.8) = 0.8
For H2,applyingReLU: ReLU(−0.1) = 0

3. Output Layer
   The activated outputs from the hidden layer are passed to the output neuron.
   The output neuron receives the values from the hidden layer neurons and computes the final prediction using weights:

   Suppose the output weights from hidden layer to output neuron are [0.7, 0.2].

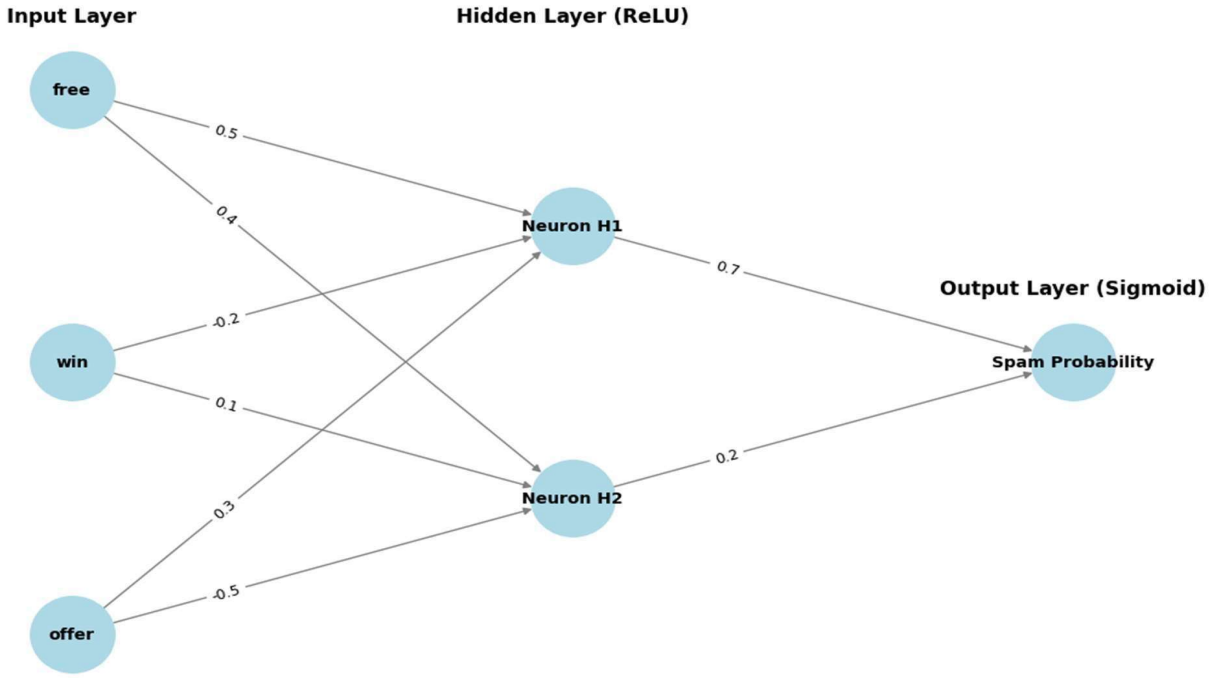
   Calculation:

   Input = (0.8×0.7) + (0×0.2) = 0.56+0 = 0.56

   Final Activation: The output is passed through a sigmoid activation function to obtain a probability:

   $\sigma(0.56) \approx 0.636$

4. Final Classification
   The output value of approximately 0.636 indicates the probability of the email being spam. Since this value is greater than 0.5, the neural network classifies the email as spam (1).



4. Learning of Approaches

1. Learning with Supervised Learning:
   In supervised learning, a neural network learns from labeled input-output pairs provided by a teacher. The network generates outputs based on inputs, and by comparing these outputs to the known desired outputs, an error signal is created. The network iteratively adjusts its parameters to minimize errors until it reaches an acceptable performance level.

2. Learning with Unsupervised Learning
   Unsupervised learning involves data without labeled output variables. The primary goal is to understand the underlying structure of the input data (X). Unlike supervised learning, there is no instructor to guide the process. Instead, the focus is on modeling data patterns and relationships, with techniques like clustering and association commonly used.

3. Learning with Reinforcement Learning
   Reinforcement learning enables a neural network to learn through interaction with its environment. The network receives feedback in the form of rewards or penalties, guiding it to find an optimal policy or strategy that maximizes cumulative rewards over time. This approach is widely used in applications like gaming and decision-making.

5. Types of Neural Networks

There are seven types of neural networks that can be used.

- Feedforward Networks: A feedforward neural network is a simple artificial neural network architecture in which data moves from input to output in a single direction.
- Convolutional Neural Network (CNN): A Convolutional Neural Network (CNN) is a specialized artificial neural network designed for image processing. It employs convolutional layers to automatically learn hierarchical features from input images, enabling active image recognition and classification.
- Single layer Perceptron: A single-layer perceptron consists of only one layer of neurons . It takes inputs, applies weights, sums them up, and uses an activation function to produce an output.
- Multilayer Perceptron (MLP): MLP is a type of feedforward neural network with three or more layers, including an input layer, one or more hidden layers, and an output layer. It uses nonlinear activation functions.
- Recurrent Neural Network (RNN): An artificial neural network type intended for sequential data processing is called a Recurrent Neural Network (RNN). It is appropriate for applications where contextual dependencies are critical, such as time series prediction and natural language processing, since it makes use of feedback loops, which enable information to survive within the network.
- Long Short-Term Memory (LSTM): LSTM is a type of RNN that is designed to overcome the vanishing gradient problem in training RNNs. It uses memory cells and gates to selectively read, write, and erase information.

Feedforward Neural Network

Feedforward Neural Network (FNN) is a type of artificial neural network in which information flows in a single direction from the input layer through hidden layers to the output layer without loops or feedback. It is mainly used for pattern recognition tasks like image and speech classification.

For example, in credit scoring system banks use an FNN which analyze users' financial profiles such as income, credit history and spending habits—to determine their creditworthiness.

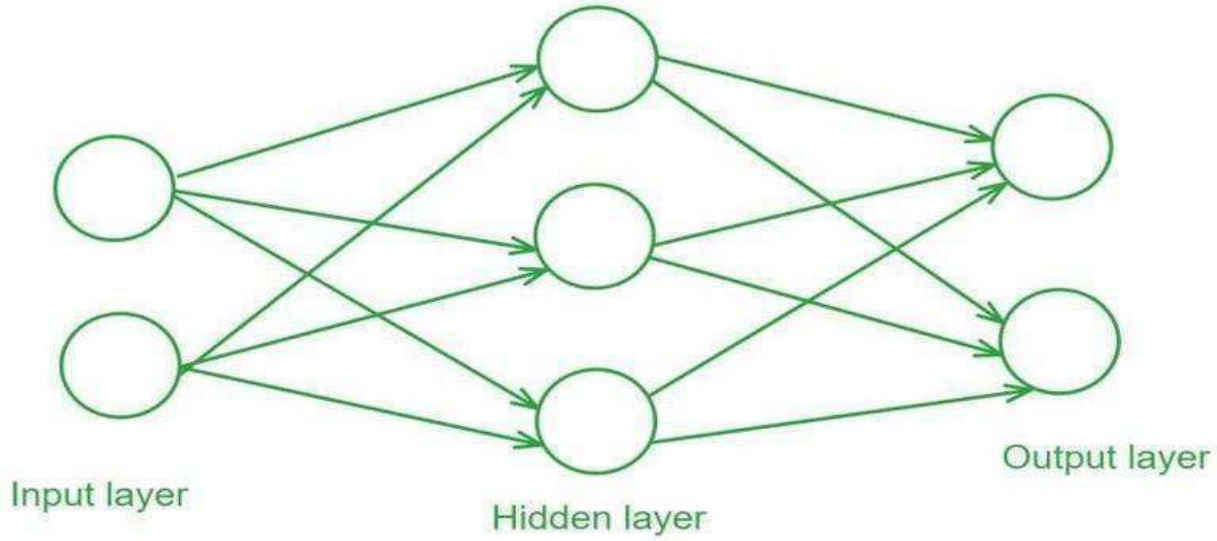
Each piece of information flows through the network's layers where various calculations are made to produce a final score

Structure of a Feedforward Neural Network

Feedforward Neural Networks have a structured layered design where data flows sequentially through each layer.

1. Input Layer: The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
2. Hidden Layers: One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.
3. Output Layer: The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.



Activation Functions

Activation functions introduce non-linearity into the network enabling it to learn and model complex data patterns.

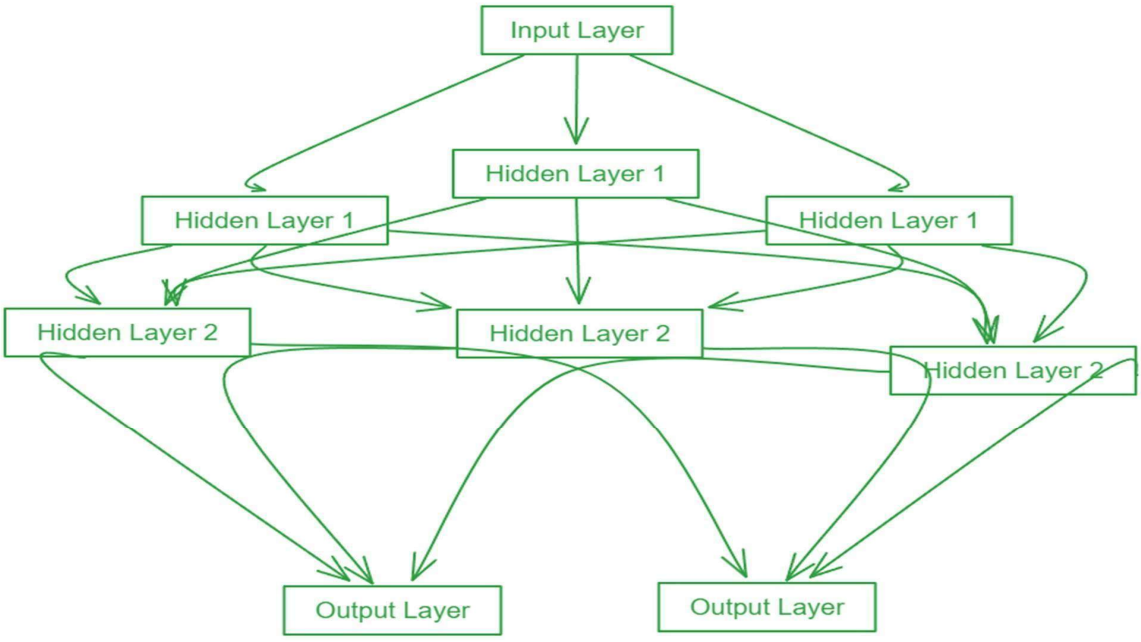
Common activation functions include:

- Sigmoid: $\sigma(x) = \sigma(x) = 11+ e - x$
- Tanh: $\tanh(x)= ex – e - x$
- ReLU: $\text{ReLU}(x)=\max(0,x)$

Training a Feedforward Neural Network

Training a Feedforward Neural Network involves adjusting the weights of the neurons to minimize the error between the predicted output and the actual output. This process is typically performed using backpropagation and gradient descent.

1. Forward Propagation: During forward propagation the input data passes through the network and the output is calculated.
2. Loss Calculation: The loss (or error) is calculated using a loss function such as Mean Squared Error (MSE) for regression tasks or Cross-Entropy Loss for classification tasks.
3. Backpropagation: In backpropagation the error is propagated back through the network to update the weights. The gradient of the loss function with respect to each weight is calculated and the weights are adjusted using gradient descent.



Gradient Descent

Gradient Descent is an optimization algorithm used to minimize the loss function by iteratively updating the weights in the direction of the negative gradient. Common variants of gradient descent include:

- Batch Gradient Descent: Updates weights after computing the gradient over the entire dataset.
- Stochastic Gradient Descent (SGD): Updates weights for each training example individually.
- Mini-batch Gradient Descent: It Updates weights after computing the gradient over a small batch of training examples.

Evaluation of Feedforward neural network

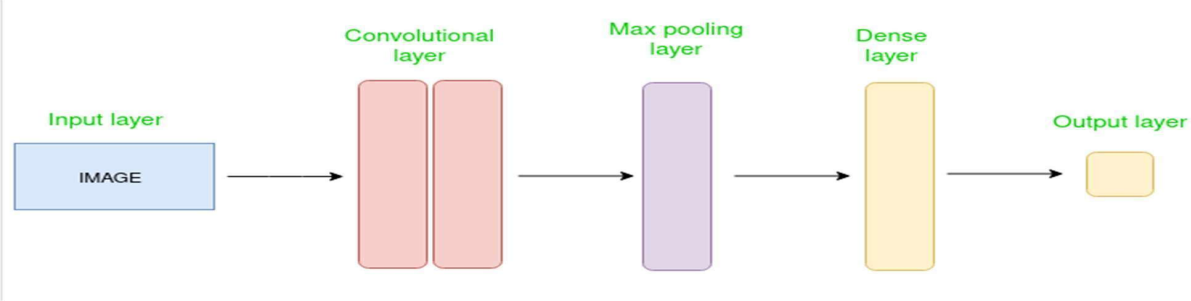
Evaluating the performance of the trained model involves several metrics:

- Accuracy: The proportion of correctly classified instances out of the total instances.
- Precision: The ratio of true positive predictions to the total predicted positives.
- Recall: The ratio of true positive predictions to the actual positives. F1 Score: The harmonic means of precision and recall, providing a balance between the two.
- Confusion Matrix: A table used to describe the performance of a classification model, showing the true positives, true negatives, false positives, and false negatives.

Introduction to Convolution Neural Network

Convolutional Neural Network (CNN) is an advanced version of artificial neural networks (ANNs), primarily designed to extract features from grid-like matrix datasets. This is particularly useful for visual datasets such as images or videos, where data patterns play a crucial role. CNNs are widely used in computer vision applications due to their effectiveness in processing visual data.

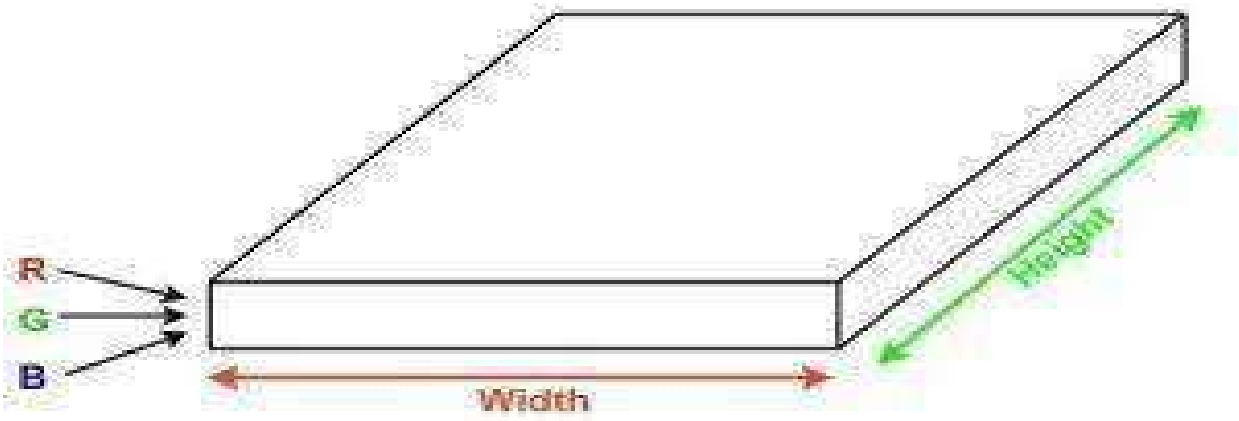
CNNs consist of multiple layers like the input layer, Convolutional layer, pooling layer, and fully connected layers. Let's learn more about CNNs in detail.



How Convolutional Layers Works?

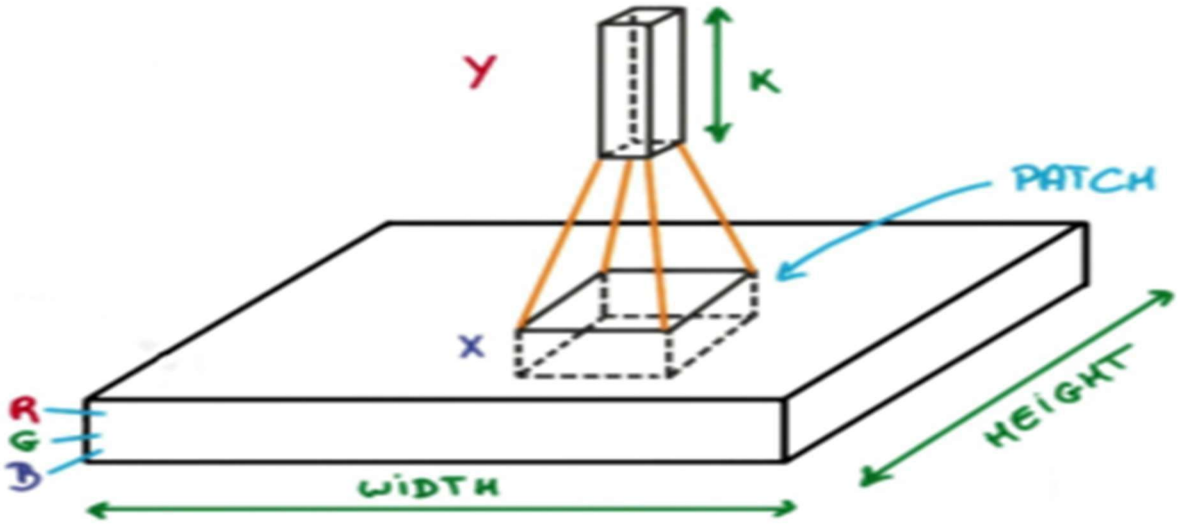
Convolution Neural Networks are neural networks that share their parameters.

Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally has red, green, and blue channels).



Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say, K outputs and representing them vertically.

Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called Convolution. If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.



Mathematical Overview of Convolution

Now let's talk about a bit of mathematics that is involved in the whole convolution process.

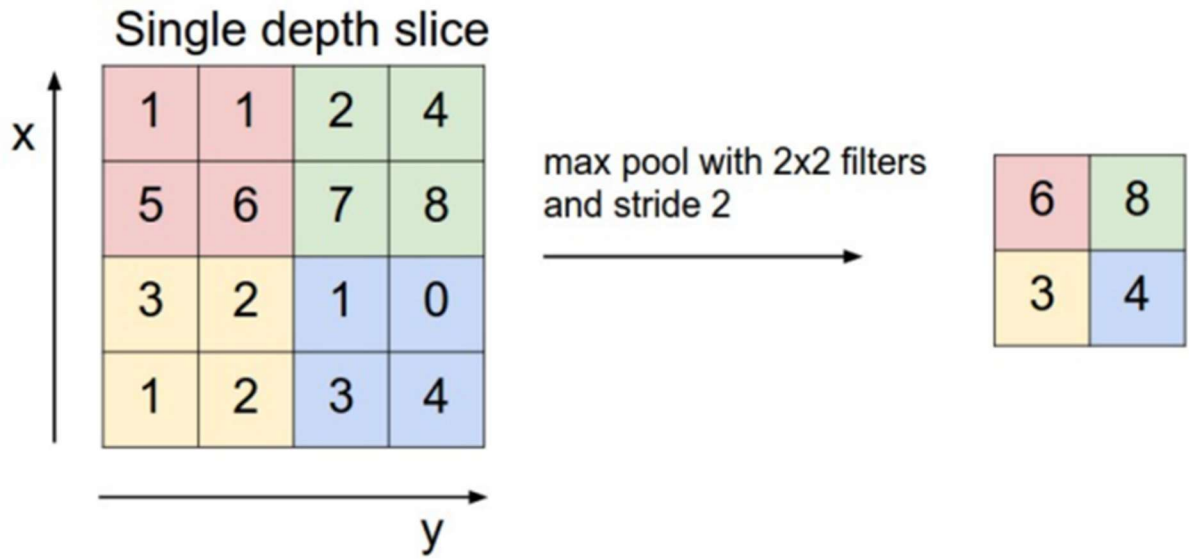
- Convolution layers consist of a set of learnable filters (or kernels) having small widths and heights and the same depth as that of input volume (3 if the input layer is image input).
- For example, if we have to run convolution on an image with dimensions 34x34x3. The possible size of filters can be axax3, where 'a' can be anything like 3, 5, or 7 but smaller as compared to the image dimension.
- During the forward pass, we slide each filter across the whole input volume step by step where each step is called stride (which can have a value of 2, 3, or even 4 for high dimensional images) and compute the dot product between the kernel weights and patch from input volume.
- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

Layers Used to Build ConvNets

A complete Convolution Neural Networks architecture is also known as covnets. A covnets is a sequence of layers, and every layer transforms one volume to another through a di erentiable function.

Let's take an example by running a covnets on of image of dimension 32 x 32 x 3.

- Input Layers: It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- Convolutional Layers: This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2x2, 3x3, or 5x5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension 32 x 32 x 12.
- Activation Layer: By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are RELU: max(0, x), Tanh, Leaky RELU, etc. The volume remains unchanged hence output volume will have dimensions 32 x 32 x 12.
- Pooling layer: This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are max pooling and average pooling. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.

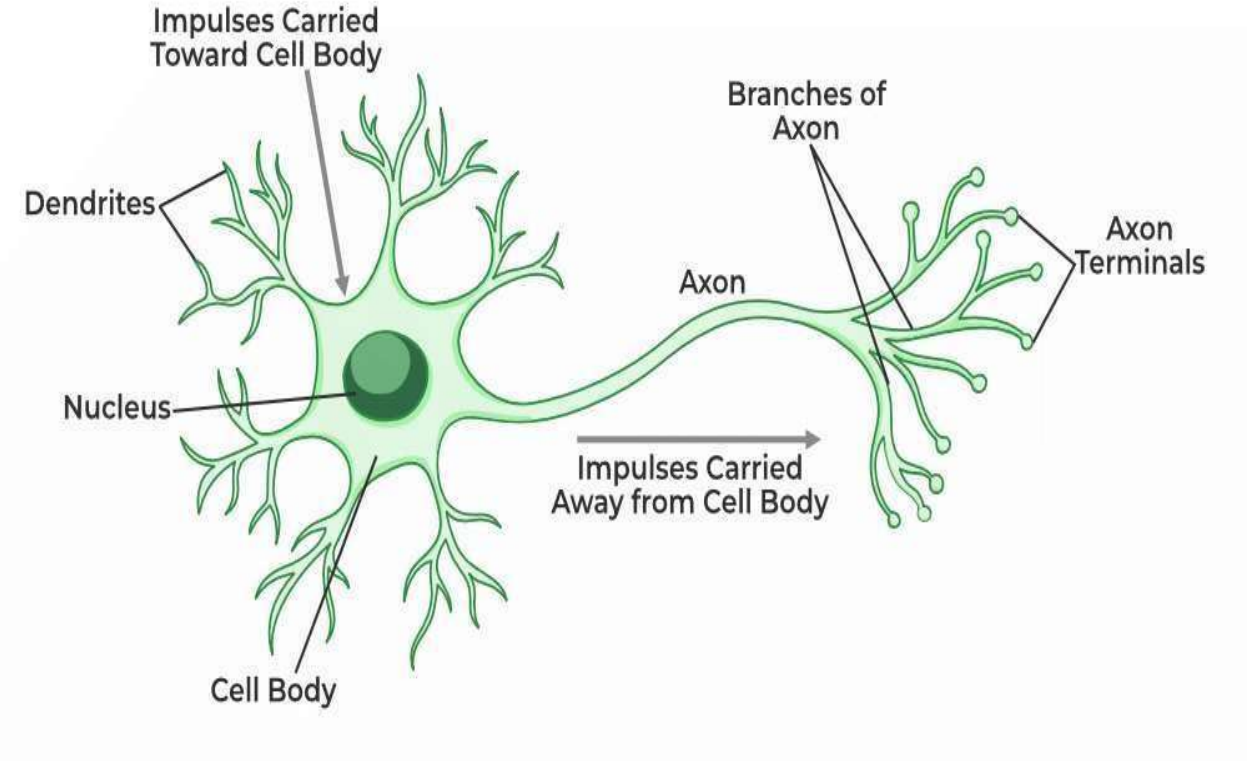


- Flattening: The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- Fully Connected Layers: It takes the input from the previous layer and computes the final classification or regression task.

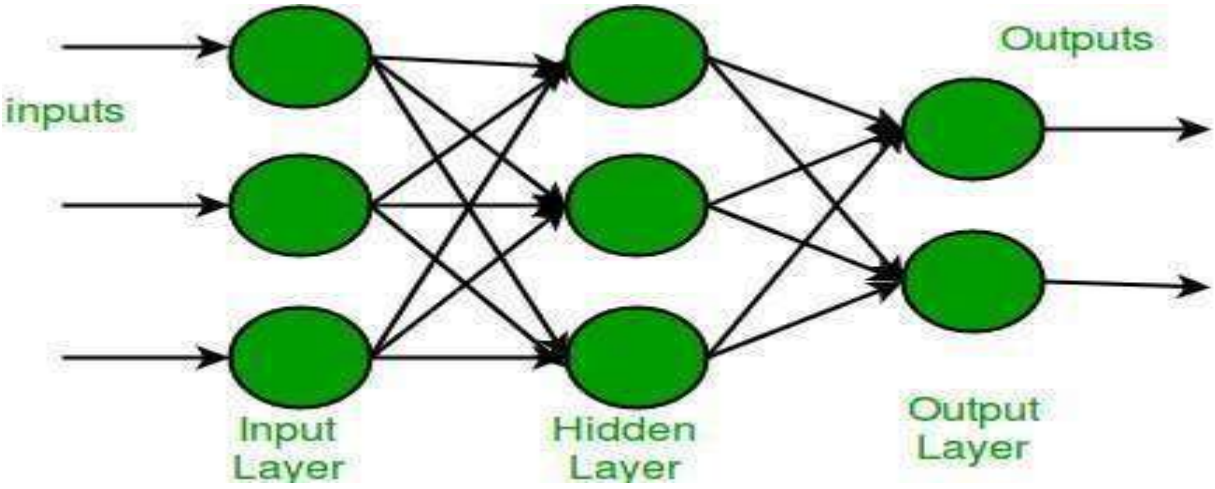
Single Layer Perceptron in TensorFlow

Single Layer Perceptron is inspired by biological neurons and their ability to process information. To understand the SLP we first need to break down the workings of a single artificial neuron which is the fundamental building block of neural networks. An artificial neuron is a simplified computational model that mimics the behavior of a biological neuron. It takes inputs, processes them and produces an output. Here's how it works step by step:

- Receive signal from outside.
- Process the signal and decide whether we need to send information or not.
- Communicate the signal to the target cell, which can be another neuron or gland.



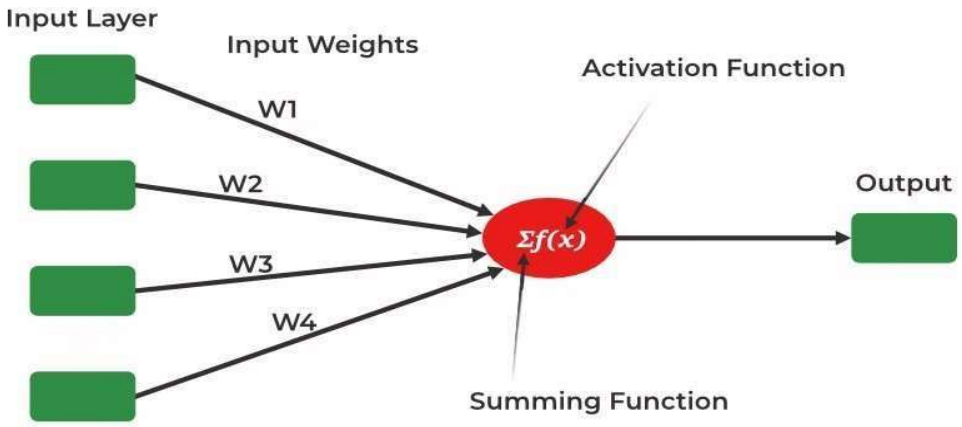
In the same way neural networks also work



Single Layer Perceptron

It is one of the oldest and first introduced neural networks. It was proposed by Frank Rosenblatt in 1958. Perceptron is also known as an artificial neural network. Perceptron is mainly used to compute the logical gate like AND, OR and NOR which has binary input and binary output. The main functionality of the perceptron is:- Takes input from the input layer - Weight them up and sum it up.

Pass the sum to the nonlinear function to produce the output.



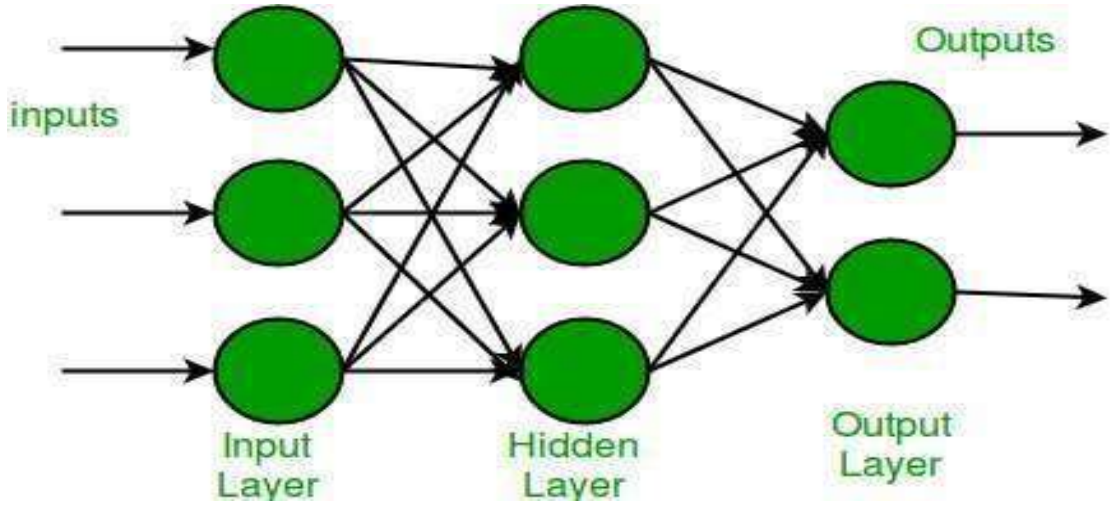
Here activation functions can be anything like sigmoid, tanh, relu based on the requirement we will be choosing the most appropriate nonlinear activation function to produce the better result.
Now let us implement a single-layer perceptron.

Multi-Layer Perceptron Learning in TensorFlow

Multi-Layer Perceptron (MLP) consists of fully connected dense layers that transform input data from one dimension to another. It is called multilayer because it contains an input layer, one or more hidden layers and an output layer. The purpose of an MLP is to model complex relationships between inputs and outputs.

Components of Multi-Layer Perceptron (MLP)

- Input Layer: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- Hidden Layers: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- Output Layer: The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.



Every connection in the diagram is a representation of the fully connected nature of an MLP. This means that every node in one layer connects to every node in the next layer. As the data moves through the network each layer transforms it until the final output is generated in the output layer.

Working of Multi-Layer Perceptron

Let's see working of the multi-layer perceptron. The key mechanisms such as forward propagation, loss function, backpropagation and optimization.

1. Forward Propagation
   In forward propagation the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

2. Weighted Sum: The neuron computes the weighted sum of the inputs:

   $z=\sum_i w_i x_i + b$

   Where:
   - x_i is the input feature.
   - w_i is the corresponding weight.
   - b is the bias term.

3. Activation Function: The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include:
   - Sigmoid: $\sigma(z)=11+e-z$

ReLU (Rectified Linear Unit): $f(z)=\max(0,z)$

Tanh (Hyperbolic Tangent): $\tanh(z)=21+e-2z-1$ 2. Loss Function

Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.

For a classification problem the commonly used binary cross-entropy loss function is:

$$L=-1N\sum i=1N[yilog(y^i)+(1-yi)log(1-y^i)]$$

Where:

- Yi is the actual label.
- y^i is the predicted label.
- N is the number of samples.

For regression problems the mean squared error (MSE) is often used:

$$MSE=1N\sum i=1N(yi-y^i)2$$

4. Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through backpropagation:

- Gradient Calculation: The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
- Error Propagation: The error is propagated back through the network, layer by layer.
- Gradient Descent: The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w = w-\eta\cdot\partial L\ \partial w$ Where:
- w is the weight.
- η is the learning rate.
- $\partial L\ \partial w$ is the gradient of the loss function with respect to the weight.

5. Optimization

MLPs rely on optimization algorithms to iteratively refine the weights and biases during training. Popular optimization methods include:

- Stochastic Gradient Descent (SGD): Updates the weights based on a single sample or a small batch of data: $w = w-\eta\cdot\partial L\ \partial w$
- Adam Optimizer: An extension of SGD that incorporates momentum and adaptive learning rates for more efficient training:

- $m_t = \beta_1 m_{t-1} + (1-\beta_1) \cdot g_t$
- $v_t = \beta_2 v_{t-1} + (1-\beta_2) \cdot g_t^2$

- Here g_t represents the gradient at time t and β_1, β_2 are decay rates.

Now that we are done with the theory part of multi-layer perception, let's go ahead and implement code in python using the TensorFlow library.

Introduction to Recurrent Neural Networks

Recurrent Neural Networks (RNNs) di er from regular neural networks in how they process information. While standard neural networks pass information in one direction i.e from input to output, RNNs feed information back into the network at each step.

Lets understand RNN with a example:

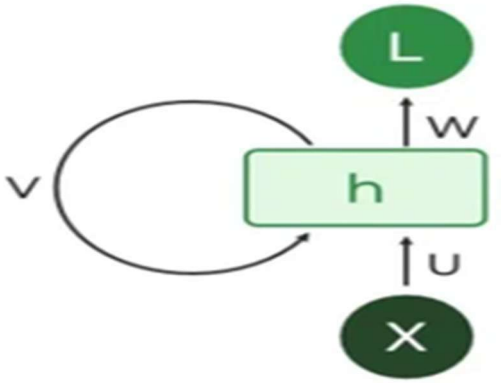
Imagine reading a sentence and you try to predict the next word, you don't rely only on the current word but also remember the words that came before. RNNs work similarly by "remembering" past information and passing the output from one step as input to the next i.e it considers all the earlier words to choose the most likely next word. This memory of previous steps helps the network understand context and make better predictions.

Key Components of RNNs

There are mainly two components of RNNs that we will discuss.

1. Recurrent Neurons

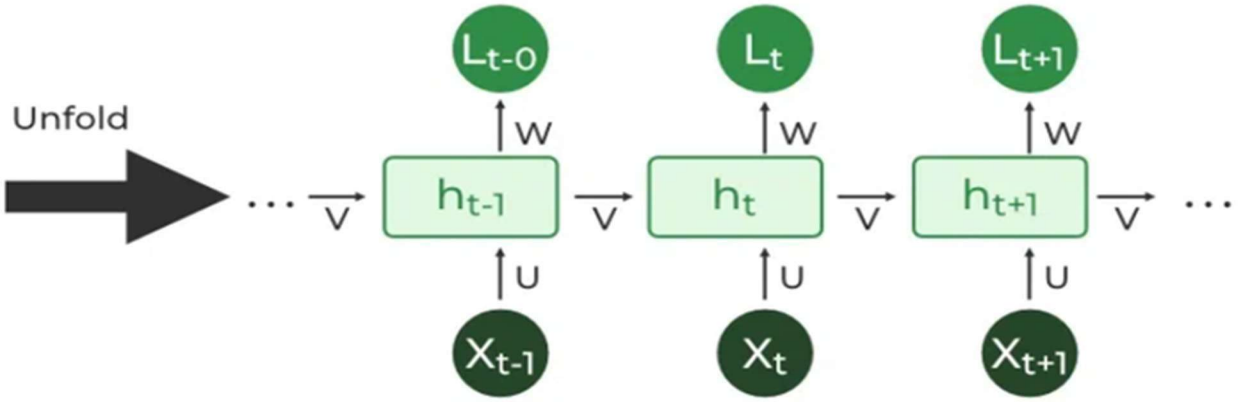
The fundamental processing unit in RNN is a Recurrent Unit. They hold a hidden state that maintains information about previous inputs in a sequence. Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



2. RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.



Recurrent Neural Network Architecture

RNNs share similarities in input and output structures with other deep learning architectures but di er significantly in how information flows from input to output. Unlike traditional deep neural networks where each dense layer has distinct weight matrices. RNNs use shared weights across time steps, allowing them to remember information over sequences.

In RNNs the hidden state Hi is calculated for every input Xi to retain sequential dependencies. The computations follow these core formulas:

1. Hidden State Calculation:

$h=\sigma(U\cdot X+W\cdot ht-1+B))$ Here:

- h represents the current hidden state.
- U and W are weight matrices.
- B is the bias.

2. Output Calculation:

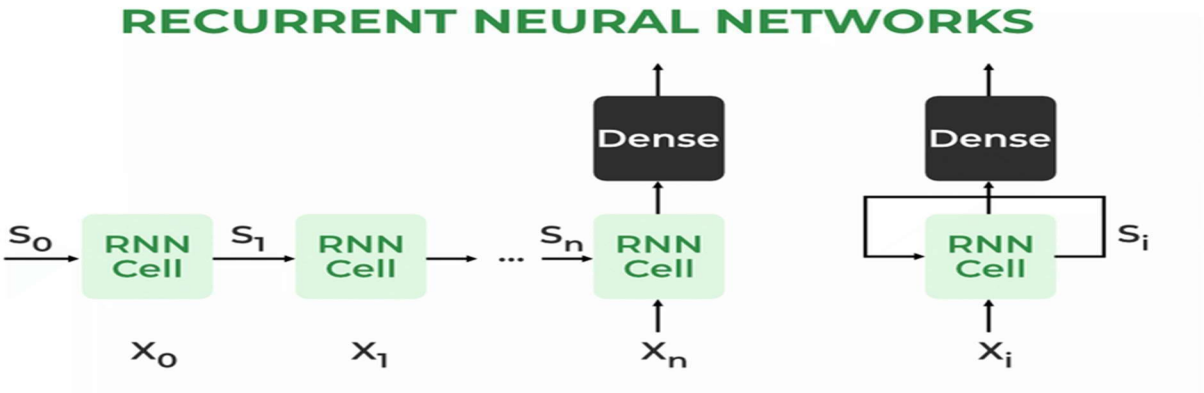
$Y=O(V\cdot h+C)$

The output Y is calculated by applying O an activation function to the weighted hidden state where V and C represent weights and bias.

3. Overall Function:

Y=f(X,h,W,U,V,B,C)

This function defines the entire RNN operation where the state matrix SS holds each element is representing the network's state at each time step ii.

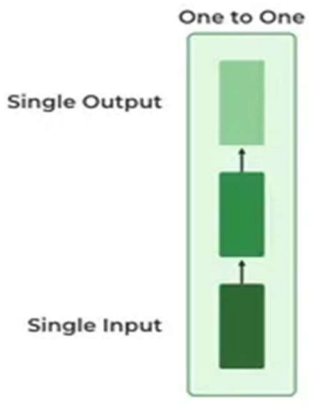


Types Of Recurrent Neural Networks

There are four types of RNNs based on the number of inputs and outputs in the network:

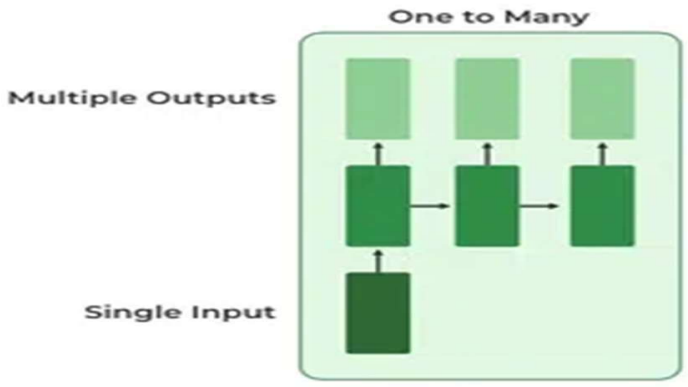
1. One-to-One RNN

This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



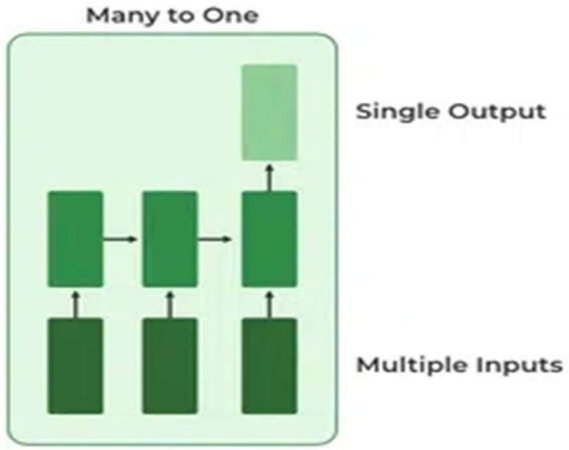
2. One-to-Many RNN

In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example in image captioning a single image can be used as input to generate a sequence of words



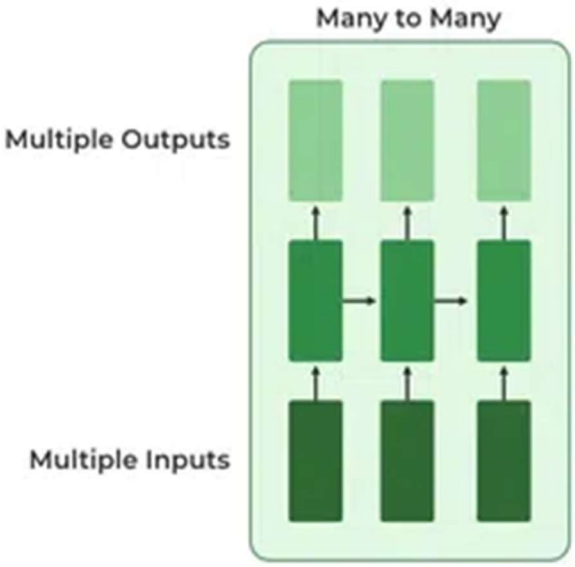
3. Many-to-One RNN

The Many-to-One RNN receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



4. Many-to-Many RNN

The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



Long Short-Term Memory:

Long Short-Term Memory (LSTM) is an enhanced version of the Recurrent Neural Network (RNN) designed by Hochreiter and Schmid Huber. LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting.

Unlike traditional RNNs which use a single hidden state passed through time LSTMs introduce a memory cell that holds information over extended periods addressing the challenge of learning long-term dependencies.

Problem with Long-Term Dependencies in RNN

Recurrent Neural Networks (RNNs) are designed to handle sequential data by maintaining a hidden state that captures information from previous time steps. However they often face challenges in learning long-term dependencies where information from distant time steps becomes crucial for making accurate predictions for current state. This problem is known as the vanishing gradient or exploding gradient problem.

- Vanishing Gradient: When training a model over time, the gradients which help the model learn can shrink as they pass through many steps. This makes it hard for the model to learn long-term patterns since earlier information becomes almost irrelevant.
- Exploding Gradient: Sometimes gradients can grow too large causing instability. This makes it difficult for the model to learn properly as the updates to the model become erratic and unpredictable.

Both issues make it challenging for standard RNNs to effectively capture long-term dependencies in sequential data.

LSTM Architecture

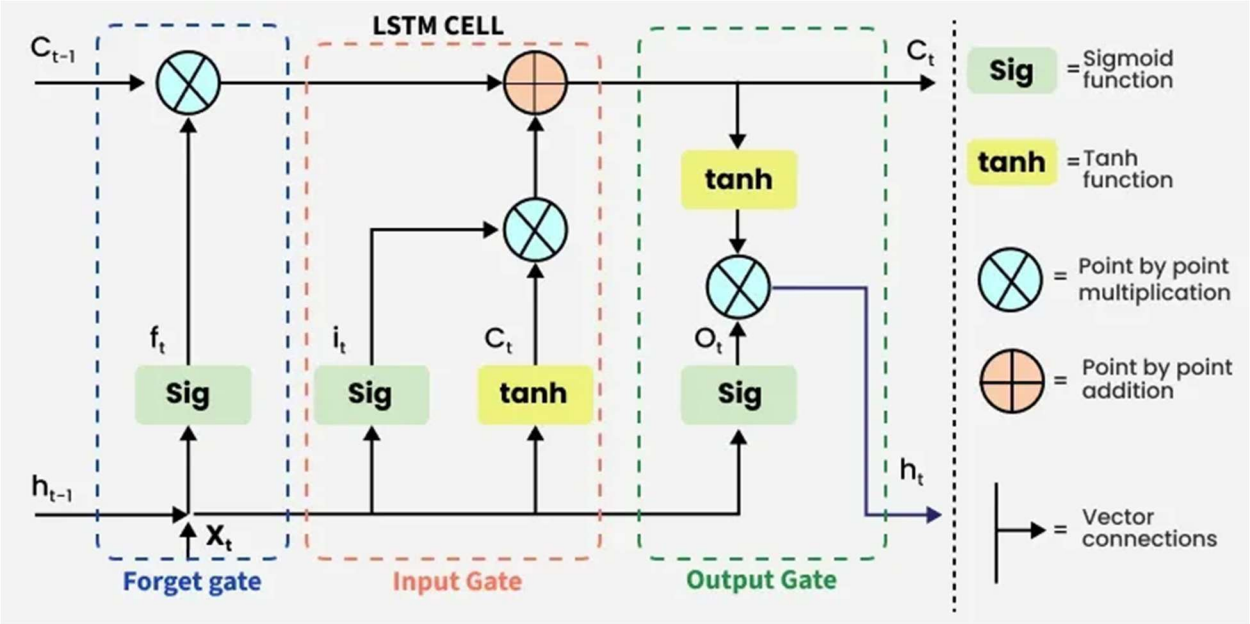
LSTM architectures involve the memory cell which is controlled by three gates:

1. Input gate: Controls what information is added to the memory cell.
2. Forget gate: Determines what information is removed from the memory cell.
3. Output gate: Controls what information is output from the memory cell.

This allows LSTM networks to selectively retain or discard information as it flows through the network which allows them to learn long-term dependencies. The network has a hidden state which is like its short-term memory. This memory is updated using the current input, the previous hidden state and the current state of the memory cell.

Working of LSTM

LSTM architecture has a chain structure that contains four neural networks and di erent memory blocks called cells.



Information is retained by the cells and the memory manipulations are done by the gates. There are three gates -

1. Forget Gate

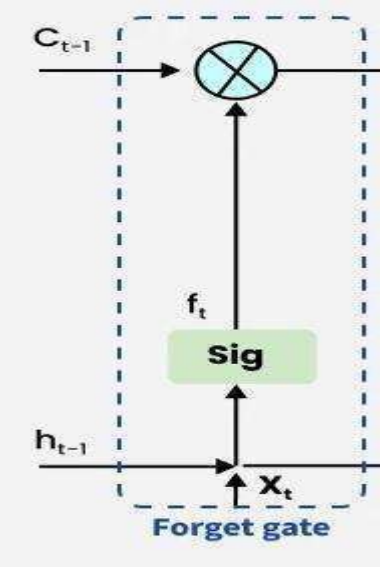
The information that is no longer useful in the cell state is removed with the forget gate. Two inputs xt (input at the particular time) and ht−1 (previous cell output) are fed to the gate and multiplied

with weight matrices followed by the addition of bias. The resultant is passed through an activation function which gives a binary output. If for a particular cell state the output is 0, the piece of information is forgotten and for output 1, the information is retained for future use.

The equation for the forget gate is:

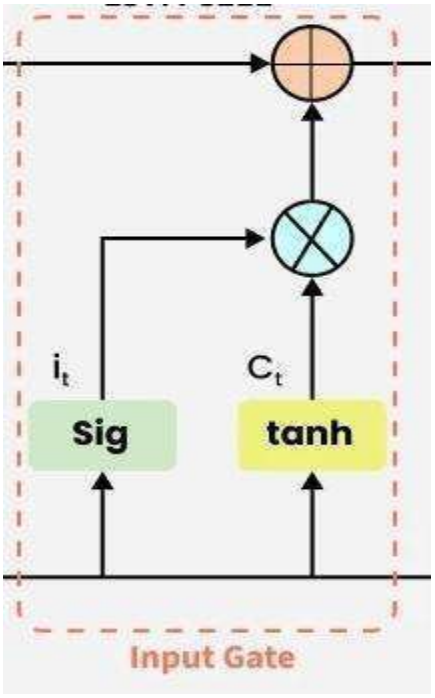
$f_t=\sigma(W_f\cdot[h_{t-1},x_t]+b_f)$ Where:

- W_f represents the weight matrix associated with the forget gate.
- $[h_{t-1},x_t]$ denotes the concatenation of the current input and the previous hidden state.
- B_f is the bias with the forget gate.
- $\sigma$is the sigmoid activation function.



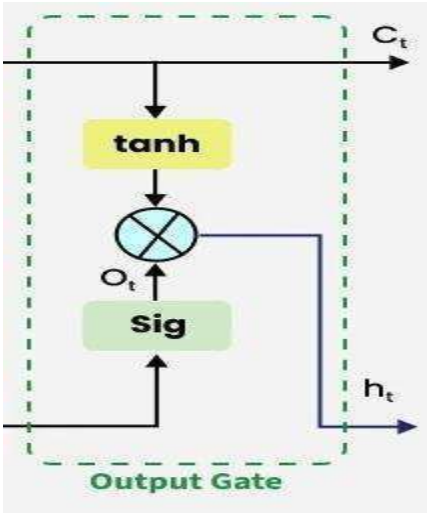
2. Input gate

The addition of useful information to the cell state is done by the input gate. First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs $-1$and x_t. Then, a vector is created using tanh function that gives an output from -1 to +1 which contains all the possible values from h_{t-1} and x_t. At last the values of the vector and the regulated values are multiplied to obtain the useful information.



3. Output gate

The task of extracting useful information from the current cell state to be presented as output is done by the output gate. First, a vector is generated by applying tanh function on the cell. Then, the information is regulated using the sigmoid function and filter by the values to be remembered using inputs ht−1and xt. At last the values of the vector and the regulated values are multiplied to be sent as an output and input to the next cell. The equation for the output gate is: ot=σ(Wo·[ht−1,xt]+bo)



Applications of LSTM

Some of the famous applications of LSTM includes:

- Language Modeling: Used in tasks like language modeling, machine translation and text summarization. These networks learn the dependencies between words in a sentence to generate coherent and grammatically correct sentences.
- Speech Recognition: Used in transcribing speech to text and recognizing spoken commands. By learning speech patterns, they can match spoken words to corresponding text.
- Time Series Forecasting: Used for predicting stock prices, weather and energy consumption. They learn patterns in time series data to predict future events.
- Anomaly Detection: Used for detecting fraud or network intrusions. These networks can identify patterns in data that deviate drastically and flag them as potential anomalies.
- Recommender Systems: In recommendation tasks like suggesting movies, music and books. They learn user behavior patterns to provide personalized suggestions.
- Video Analysis: Applied in tasks such as object detection, activity recognition and action classification. When combined with Convolutional Neural Networks (CNNs) they help analyze video data and extract useful information.

References

https://www.w3schools.com/ai/
https://www.geeksforgeeks.org/machine-learning/neural-networks-a-beginners-guide/ https://www.tutorialspoint.com/artificial_neural_network/index.htm
https://www.datacamp.com/blog/what-are-neural-networks